

백엔드 개발자

홍철민

실패 가능성을 전제로 시스템의 상태와 책임 경계를 설계합니다.

Java/Kotlin-Spring 기반 백엔드 경험을 바탕으로 결제 상태, 비동기 메시징, 대용량 파일 처리
와 AI 서비스 백엔드를 다뤘습니다.

- Java · Kotlin
- Spring
- MySQL · Redis
- AWS
- Python · MCP

- 01

호두랩스

주문을 REQUESTED·PURCHASED·COMPLETED로 나누고, PURCHASED 상태와 Redis 정보가 남은 주문만 복구 배치가 재처리하도록 했습니다.
- 02

WorkShield Web — 실패 경계를 설계한 AI 서비스 백엔드

Review Aggregate에 상태 전이를 모으고 멍등 키·DB 제약·낙관적 잠금을 서로 다른 방어선으로 사용했으며, LLM이 아닌 API가 질문 대상과 출처를 확정하도록 했습니다.
- 03

KExcel — Kotlin DSL 기반 대용량 엑셀 생성 라이브러리

사용자 DSL과 실제 엑셀 엔진을 ExcelDriver 인터페이스로 분리하고, 스트리밍 제약과 동일 시트 동시 쓰기 오류는 즉시 실패하도록 했습니다.

호두랩스 통합 결제

외부 결제 승인과 내부 재화 제공이 한 번에 끝나지 않을 수 있어, 승인 이후 내부 처리가 완료되지 않은 주문을 구분할 필요가 있었습니다.

2021.02-2021.08

결제 API 및 복구 배치 개발

Java · Spring Boot · MySQL · Redis · ShedLock

REQUESTED

주문·주문키 생성



외부 승인

PURCHASED

승인 DB 반영



재화 제공

COMPLETED

내부 처리 완료

핵심 판단

주문을 REQUESTED·PURCHASED·COMPLETED로 나누고, PURCHASED 상태와 Redis 정보가 남은 주문만 복구 배치가 재처리 하도록 했습니다.

중복 방어

- 처리 전 DB 주문 상태 검증
- 결제 트랜잭션 식별키 UNIQUE 제약
- 애플리케이션 검증과 DB 최종 방어를 구분

역할 경계

애플리케이션 결제 흐름을 구현했으며, Stored Procedure와 DB 처리 구조는 DBA와 협업했습니다.

직접 담당

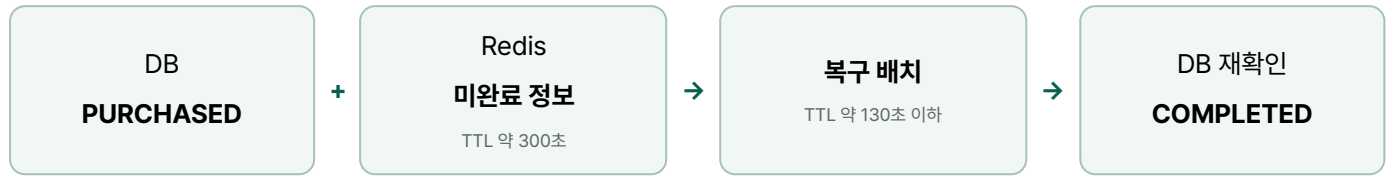
- PG 결제 API 구현
- Google Play·Apple 인앱 결제 구현
- 결제 상태 흐름 구현
- 미완료 주문 복구 배치 구현

검증 범위

주문 상태, Redis TTL과 DB 제약이 담당하는 범위를 대조해 정상 처리와 재처리 조건, 처리하지 못하는 실패를 구분했습니다.

복구 흐름과 책임 범위

외부 승인이 끝났다는 사실과 내부 재화 제공이 끝났다는 사실을 분리해, 재처리가 가능한 실패만 복구 대상으로 삼았습니다.



복구 가능한 범위

- PURCHASED 저장 후 재화 제공이 실패한 주문
- Redis 미완료 정보가 TTL 안에 남은 주문
- DB 상태 재확인 후 완료 처리가 가능한 주문

이 흐름으로 복구하지 못하는 범위

- PG 승인 후 DB의 PURCHASED 저장 자체가 실패하면 Redis와 DB 상태가 REQUESTED에 남을 수 있어 이 흐름으로 복구할 수 없습니다.
- 서비스 중단이 Redis TTL보다 길거나 캐시가 유실되면 복구 대상 정보를 찾을 수 없습니다.
- 외부 결제 상태와 내부 DB를 정기적으로 대조하는 reconciliation은 구현 범위에 없었습니다.

기술적 판단

스케줄러 중복 실행과 주문 역등성을 같은 문제로 다루지 않았습니다.

ShedLock은 여러 WAS의 복구 스케줄러가 동시에 실행되는 범위만 제한합니다. 전체 주문 처리나 외부 승인의 Exactly-once를 보장하는 장치로 확대하지 않습니다.

확인한 한계

PG 승인 후 PURCHASED 저장 자체가 실패하거나 Redis 정보가 유실·만료되면 이 복구 흐름만으로 처리할 수 없습니다.

KExcel — Kotlin DSL 기반 대용량 엑셀 생성 라이브러리

Apache POI의 저수준 API를 직접 사용하면 객체와 좌표, 스타일 생명주기를 반복해서 관리해야 했고, 엔진을 바꾸면 보고서 코드도 함께 변경해야 했습니다.



구조와 안전 제약

- 동일 DSL과 엔진 구현을 ExcelDriver로 분리
- 행 단위 스트리밍과 처리 완료 행 재접근 차단
- 동일 시트 동시 쓰기 오용을 Fail-Fast로 감지
- 엔진 고유 기능은 Native Extension으로 노출

측정 환경

OpenJDK 21, 512MB heap, G1GC, JMH 1.36에서 Native API와 DSL을 같은 데이터 규모로 비교했습니다.

Thread-safe 쓰기를 지원한다는 뜻이 아니라 데이터 손상 전에 오류를 드러내는 정책입니다.

FastExcel DSL 오버헤드

3.1%

Native API 대비

100만 행 생성

3.396s

FastExcel · 512MB heap

POI 병합 처리량

3.36 → 17.42

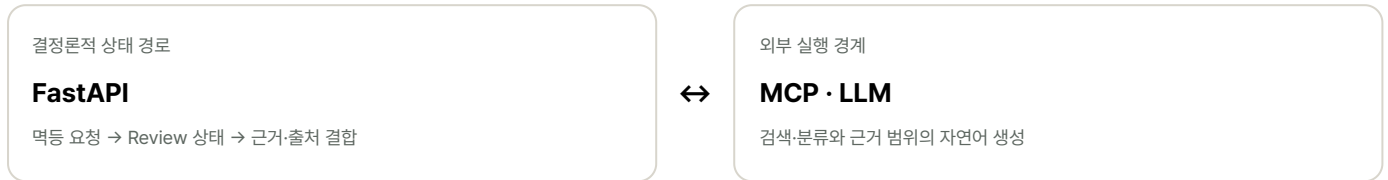
ops/s · 병합 시나리오

트레이드오프

병합 중복 검증을 우회한 최적화는 잘못된 병합 요청을 사전에 방지해야 하며, 병합 메타데이터의 메모리 비용은 남습니다.

WorkShield Web — 실패 경계를 설계한 AI 서비스 백엔드

계약서 비교 MCP를 FastAPI 웹 API와 LLM 설명 계층, AWS 배포 환경에 연결했습니다. 비동기 검토 상태를 Aggregate로 관리하고 멍등 키·UNIQUE 제약·낙관적 잠금으로 중복 요청과 상태 경쟁을 구분해 처리했습니다.



중복과 상태 경쟁

Review 상태 전이를 Aggregate에 모으고, 멍등 키·세션별 active UNIQUE 제약·낙관적 잠금을 서로 다른 방어선으로 사용했습니다.

단일 worker·SQLite 범위이며 분산 환경의 Exactly-once 실행을 의미하지 않습니다.

LLM 책임 경계

질문 대상과 근거를 AnswerPlan에서 먼저 확정하고, 모델은 출처 종류만 선택하게 했습니다. 실제 출처 ID는 백엔드가 검증된 근거에 결합합니다.

합성 fixture 최종 PASS

16 / 16

8개 × 2회 · repair 포함

모델 latency p50

7.102s

concurrency 1

모델 latency p95

16.416s

고정 합성 fixture

책임 범위

SQLite와 프로세스 내부 실행 큐, 단일 API worker·단일 EC2를 사용하는 출시 전 구조로 다중 인스턴스와 중단 지점 자동 재개를 지원하지 않습니다.

경력 요약

Java·Kotlin과 Spring 기반 애플리케이션 개발에서 시작해 결제, 비동기 메시징, 도메인 분리와 배포 환경을 다뤘습니다.

2022.11~2023.08

샤플앳컴퍼니

백엔드 개발

배치 서버와 메일 서버를 SQS로 분리하고 발송 요청과 메시지 소비 단계에 서로 다른 DB UNIQUE 제약을 적용했습니다.

배치 재실행과 메시지 재전달 시나리오를 분리해 각 UNIQUE 키와 방어 범위를 대조했습니다.

한계 · DB 제약은 확인된 두 단계의 중복 처리를 방어하지만 메일의 Exactly-once 전송이나 전체 실패 복구를 보장하지 않습니다.

2021.09~2022.10

휴니크

개발 리드·백엔드·인프라

기존 구조를 그대로 신규 시스템에 복제하지 않고 도메인별 책임을 분리한 별도 Spring 시스템을 개발했습니다. 기존 회원은 기존 인증 서비스와 연결했습니다.

주요 어드민·키오스크 API와 프론트엔드·백엔드 배포 환경까지 구현했습니다. 다만 프로젝트가 출시 전에 종료돼 실제 사용자 트래픽에서는 검증하지 못했습니다.

한계 · 운영 이전과 데이터 마이그레이션까지 완료한 사례가 아니라, 신규 시스템의 설계·구현 및 배포 환경 구성 범위를 설명하는 사례입니다.

2020.02~2021.02

지투이

애플리케이션·백엔드 개발

화면별로 반복하지 않도록 인증·세션·전역 오류·감사 로그를 공통 기능으로 분리하고 Spring Boot API와 연결했습니다.

인증 상태와 오류 처리를 공통 경로로 적용하고, 메뉴별 권한에 따라 접근 가능 범위를 구분했습니다.

한계 · 개인정보 암호화나 전체 보안 정책을 설계한 사례가 아니라, 애플리케이션 수준의 인증·세션·접근 권한 구현 사례입니다.

경력 사례 공개 원칙

회사 내부 코드와 실제 사용자 데이터는 포함하지 않았으며, 운영·출시 상태와 개인·팀의 책임 범위를 확인 가능한 수준으로 제한했습니다.

추가 프로젝트

SK Networks Family AI 캠프 팀 프로젝트

WorkShield — 표준계약서 비교 MCP

IT·SW 계약서를 표준계약서와 조항 단위로 비교해 검토 후보와 근거를 제공하는 MCP 서버를 5인 팀에서 개발했습니다. 실제 실행 점수의 범위가 모듈 간 임계값 계약과 맞지 않던 문제를 수정하고, 실행 trace와 분리된 골든 데이터로 분류 기준선을 관리했습니다.

역할

PM·시스템 설계·MCP 구현

검증

실제 실행에 사용된 검색 후보와 점수를 보존하고, 골든 데이터 135건을 tuning과 held-out으로 분리해 회귀 기준선을 관리했습니다.

한계

평가 지표는 법률적 정확도가 아니라 표준조항 대응 상태의 분류 기준선입니다.

SK Networks Family AI 캠프 팀 프로젝트

영화 흥행 예측 및 배급 시뮬레이터

스크린 수가 흥행에 영향을 주는 동시에 콘텐츠 잠재력에 따라 결정될 수 있다는 점을 고려해, 콘텐츠 잠재력과 배급 변수를 나눈 2-Stage 예측 구조를 설계했습니다. 2010~2025년 영화 2,489편을 개봉일 기준으로 분할해 모델을 비교하고, 단조 증가 제약을 적용한 최종 예측을 구현했습니다.

역할

PM·2-Stage 모델 설계

검증

2010~2025년 2,489편·28개 컬럼 데이터를 개봉일 기준으로 시간 분할해 각 Stage의 R^2 와 RMSE를 기록했습니다.

한계

관측 데이터 기반 예측 구조로, 스크린 수의 인과 효과를 검증하거나 실제 배급 의사결정을 대체하지 않습니다.

SK Networks Family AI 캠프 팀 프로젝트

전기차 충전 인프라 SOS 대시보드

공공데이터의 전기차 등록 대수와 충전기 수를 수집해 Pandera로 검증하고 MySQL에 적재한 뒤, 17개 시·도별 팀 정의 충전 불편지수를 지도와 차트로 시각화했습니다. 데이터 처리와 화면 로직은 domain-web-config 레이어로 분리했습니다.

역할

백엔드·대시보드 구현

검증

17개 시·도 단위로 전기차 등록 대수와 충전기 수를 바탕으로 계산한 불편지수를 지도와 차트로 확인했습니다.

한계

불편지수는 팀이 정의한 분석 지표이며 실제 충전 대기 시간·사용자 행동이나 공식 정책 지표를 뜻하지 않습니다.

상세 자료

각 사례의 구조, 측정 조건, 근거와 전체 한계는 HTML 포트폴리오에서 확인할 수 있습니다.

Portfolio

<https://hong1008.github.io/portfolio/>

GitHub

<https://github.com/Hong1008>

기술 블로그

<https://velog.io/@hong1008/posts>

Email

zxchnm6404@gmail.com